Research Software Engineering in Python for Reproducible Computational Science

Sylwester Arabas

AGH University of Krakow

2026-03-12

previously on...

- what (or who) is RSE?
- Python as a glue language
- Pint for unit-aware coding/plotting and dimensional analysis

plan for today

- more on reproducible computational science
- unveiling a new project: ...
- course agenda, logistics and grading
- introduction to SciPy (code along!)

plan for today

- **more on reproducible computational science**
- unveiling a new project: ...
- course agenda, logistics and grading
- introduction to SciPy (code along!)

Geoscientific Model Development journal guidelines

Geoscientific Model Development
Open Access
EGU

Geosci. Model Dev., 12, 2215–2225, 2019
https://doi.org/10.5194/gmd-12-2215-2019
© Author(s) 2019. This work is distributed under the Creative Commons Attribution 4.0 License.

Editorial: The publication of geoscientific model developments v1.2

GMD executive editors

3.1 Source code is necessary but not sufficient

In order for the reader to have some confidence that the hypotheses above hold, it is not sufficient that the source code is provided. It is also necessary to have access to all of the input data and to know all of the steps which were taken from raw data to points on graphs or numbers in tables. This also implies that all model configuration files are provided.

3.2 Manual processing considered harmful

A particular challenge to understanding and reproducing results occurs where model inputs or outputs have been manually processed by an author. This frequently occurs when figures are produced interactively. Unfortunately, this breaks the provenance chain between the model and the paper: nobody, not even the author, can definitively know what processing was done to the data and therefore how the results came about. The only remedy for this is for authors to consistently ensure that there is no manual processing of the data: models are run by a script, and all pre- and post-processing is scripted. These scripts themselves should then be archived and cited from the paper. All figures and tables must be scientifically reproducible from the scripts.

https://doi.org/10.5194/gmd-12-2215-2019

AGH

automate!

plan for today

- more on reproducible computational science
- **unveiling a new project: ...**
- course agenda, logistics and grading
- introduction to SciPy (code along!)

Handbook of **A**utomation for **BE**ng & **M**aster's projects **US**ing **PYTHON**

https://habemus-python.github.io/

plan for today

- more on reproducible computational science
- unveiling a new project: ...
- **course agenda, logistics and grading**
- introduction to SciPy (code along!)

proposed course agenda

	Lecture (×5)	Lab (×7) + Proj (×3)
week 1	RSE, unit-aware coding	
week 2	HabemusPy project; SciPy	
week 3	JIT compilation+SciPy+Pint	
week 4	packaging & archival tools	
week 5	licensing, versioning, docs	
week 6	.....	git & GitHub
week 7	.....	Python packaging & CI workflows
week 8	.....	HabemusPy contributor primer
week 9	.....	Jupyter notebooks & Quarto
week 10	.....	debugging and profiling in Python
week 11	.....	pytest & codecov; project ideas
week 12	.....	project pitches
week 13	.....	project updates (PR submission)
week 14	.....	feedback session (code reviews)
week 15	.....	**final project presentations**

AGH

grading and projects

Syllabus:

The final grade is an equal-weight mean of grades from: (i) lecture-material test, (ii) laboratory assignments and (iii) project.
A minimum of 50% of points from each component is required

grading and projects

Syllabus:

The final grade is an equal-weight mean of grades from: (i) lecture-material test, (ii) laboratory assignments and (iii) project.
A minimum of 50% of points from each component is required

- lecture-material test: CI upload of a Python package to ‘test.pypi.org‘ + Colab link (deadline: week 9)

grading and projects

Syllabus:

The final grade is an equal-weight mean of grades from: (i) lecture-material test, (ii) laboratory assignments and (iii) project.
A minimum of 50% of points from each component is required

- lecture-material test: CI upload of a Python package to ‘test.pypi.org‘ + Colab link (deadline: week 9)
- laboratory assignments: 1 PR + 1 code review for HabemusPy with curricular content (deadline: week 12)

grading and projects

Syllabus:

The final grade is an equal-weight mean of grades from: (i) lecture-material test, (ii) laboratory assignments and (iii) project.
A minimum of 50% of points from each component is required

- lecture-material test: CI upload of a Python package to ‘test.pypi.org‘ + Colab link (deadline: week 9)
- laboratory assignments: 1 PR + 1 code review for HabemusPy with curricular content (deadline: week 12)
- project: 1 pitch + 1 PR + 1 code review to HabemusPy with extracurricular content (deadilne: week 15)

AGH

plan for today

- more on reproducible computational science
- unveiling a new project: ...
- course agenda, logistics and grading
- **introduction to SciPy (code along!)**

SciPy

Fundamental algorithms for scientific computing in Python

GET STARTED

SciPy 1.17.1 released! 2026-02-22

Fundamental algorithms

SciPy provides algorithms for optimization, integration, interpolation, eigenvalue problems, algebraic equations, differential equations, statistics and many other classes of problems.

Broadly applicable

The algorithms and data structures provided by SciPy are broadly applicable across domains.

Foundational

Extends NumPy providing additional tools for array computing and provides specialized data structures, such as sparse matrices and k-dimensional trees.

Performant

SciPy wraps highly-optimized implementations written in low-level languages like Fortran, C, and C++. Enjoy the flexibility of Python with the speed of compiled code.

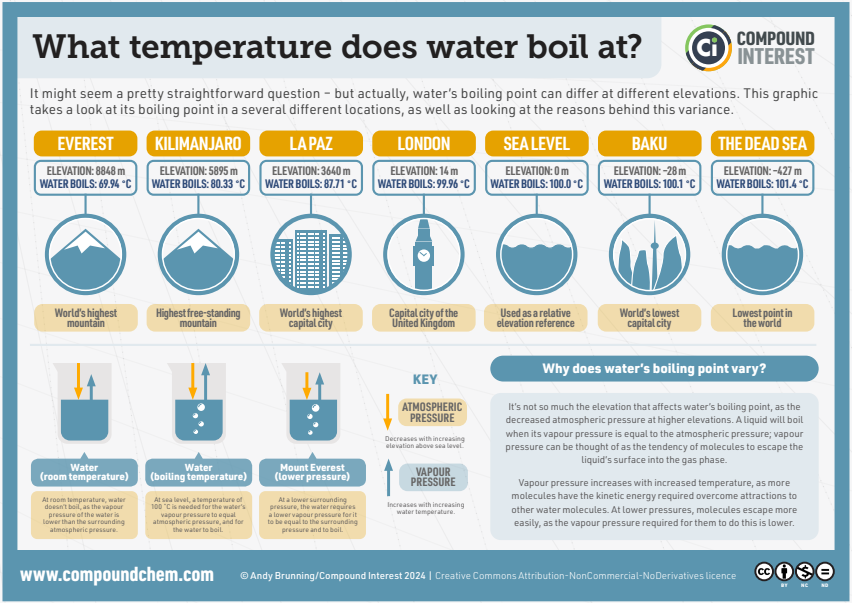
Easy to use

SciPy's high level syntax makes it accessible and productive for programmers from any background or experience level.

Open source

Distributed under a liberal BSD license, SciPy is developed and maintained publicly on GitHub by a vibrant, responsive, and diverse community.

AGH



https://compoundchem.com/2016/03/22/boiling-point/

AGH

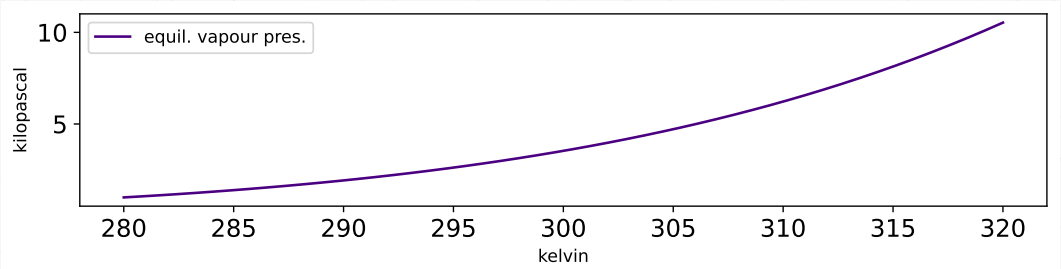
sample problem: find T_{boil} of water at a given pressure (1/4)

```python
import pint
import numpy as np
from scipy import constants

si = pint.UnitRegistry()

def equilibrium_vapour_pressure(T):
    """ Teten's formula (https://en.wikipedia.org/wiki/Tetens_equation) """
    TC = T - constants.zero_Celsius * si.K
    return .61078 * si.kPa * np.exp(17.27 * TC / (237.3 * si.K + TC))

print(f"{equilibrium_vapour_pressure(T=300 * si.K)=:.2g~}")
```

```
equilibrium_vapour_pressure(T=300 * si.K)=3.5 kPa
```

sample problem: find T_{boil} of water at a given pressure (2/4)

```
import pint
from matplotlib import pyplot
si.setup_matplotlib()

T = np.linspace(280, 320) * si.K
pyplot.plot(T, equilibrium_vapour_pressure(T), label='equil. vapour pres.')
pyplot.legend()
```

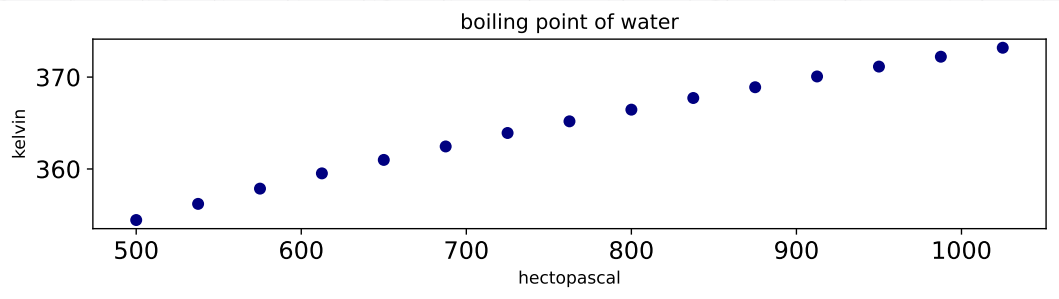


sample problem: find T_{boil} of water at a given pressure (3/4)

```python
def bisect(*, fun, x_min, x_max, atol):
    """ https://en.wikipedia.org/wiki/Bisection_method """
    fmin, fmax = fun(x_min), fun(x_max)
    while True:
        mid = (x_min + x_max) / 2
        if abs(x_max - x_min) <= atol:
            break
        fmid = fun(mid)
        if (fmid * fmin > 0):
            x_min, fmin = mid, fmid
        else:
            x_max, fmax = mid, fmid
    return mid
```

sample problem: find T_{boil} of water at a given pressure (4/4)

```
pyplot.title('boiling point of water')
for p_atm in np.linspace(500, 1025, 15) * si.hPa:
    T_boil = bisect(fun=lambda T: equilibrium_vapour_pressure(T) - p_atm,
                    x_min=300 * si.K, x_max=400 * si.K, atol=.1 * si.K)
    pyplot.scatter(p_atm, T_boil, color='navy')
```



reuse!

could we use SciPy?

```
from scipy.optimize import root_scalar

try:
    res = root_scalar(lambda T: equilibrium_vapour_pressure(T) - p_atm,
        bracket=[300 * si.K, 400 * si.K], method='bisect', xtol=0.1 * si.K)
except Exception as e:
    print(str(e))
```

```
Unable to parse arguments
```

what's wrong?!

ChatGPT

Log in | Sign up for free

Hi, I'm trying to use scipy.optimize.root_scalar the following way (where si is an instance of pint.UnitRegistry):

```
res = root_scalar(lambda T: equilibrium_vapour_pressure(T) - p_atm,
    bracket=[300 * si.K, 400 * si.K], method='bisect', xtol=0.1 * si.K)
```

but it raises an exception with "Unable to parse arguments" message, how to fix it?

Ah! The issue here is that `scipy.optimize.root_scalar` **does not natively understand Pint quantities with units**. It expects **plain Python floats** for `x0`, `x1`, `bracket`, and `xtol`. When you pass `300 * si.K` or `0.1 * si.K`, it raises the "Unable to parse arguments" error because it tries to convert a Pint `Quantity` to a float internally and fails.

Here's how you can fix it:

---

1. Strip units before passing to `root_scalar`

let's solve it properly next week!

Thank you for your attention!

slides and other course resources at
https://github.com/open-atmos-krk/rse-python-course